

2027 考研数据结构代码题 基础 48 题

b 站/公众号/小红书同名：我头发还多还能学

微信：Rain-Splash petrichoryin bubumelody（加一个即可）

课程介绍网站：toufaduokaoyan.com

强化篇、真题篇以及对应的视频讲解需付费，有需要的添加上方微信即可

打印资料可以直接扫右侧二维码，价格无敌便宜，纸张质量也很棒

5分/页



扫码打印 首单免费

基础篇目录

顺序表

- 1、在顺序表 L 的第 k 个位置插入元素 x
- 2、将顺序表中的元素逆置
- 3、查找顺序表中第一个值为 x 的元素的位置
- 4、删除顺序表中值为 x 的元素（仅有一个 x）
- 5、顺序表递增有序，插入元素 x，仍递增有序（非递减）
- 6、删除整个顺序表中最小的元素

链表：

- 1、对带头结点的单链表 L 头插/尾插一个值为 x 的节点
- 2、查找单链表中第一个值为 x 的结点
- 3、试编写算法将带头结点的单链表就地逆置
- 4、删除单链表中第一个值为 x 的结点
- 5、对递增单链表 L，插入一个值为 x 的结点后。使其保持非递减有序
- 6、删除单链表 L 中最小值结点（多个选择第一个/默认唯一）
- 7、删除循环单链表 L 中最小值结点（多个选择第一个/默认唯一）
- 8、按递增有序地输出单链表中各结点的数值，并释放结点所占空间
- 9、将一个带头节点双向链表中值最小的结点插入到整个链表的最前面

栈和队列：

- 1、判断字符串是否回文
- 2、判断单链表中全部 n 个字符是否回文
- 3、判断一个表达式中括号是否配对
- 4、判断子串 s2 是否匹配母串 s1，若匹配，输出匹配到第一个字符的索引
- 5、有两个链表 A 和 B，判断 B 是否是 A 的连续子序列

树

- 1、计算二叉树中所有结点个数
- 2、计算二叉树中所有叶子结点的个数
- 3、计算二叉树中所有双分支的结点个数
- 4、找出二叉树中值最大的结点
- 5、把二叉树所有结点左右子树交换

- 6、求二叉树中值为 x 的层号
- 7、计算二叉树的最大深度（高度）
- 8、判断两个二叉树是否相似
- 9、求树的度（树中结点度取最大值为树的度）
- 10、判断二叉树是不是正则二叉树（即每个结点的度均为 0 或 2）
- 11、层次遍历二叉树

图

- 1、实现连通图 G（以邻接表存储）的广度优先遍历（BFS）
- 2、实现连通图 G（以邻接矩阵存储）的广度优先遍历（BFS）
- 3、实现连通图 G（以邻接表存储）的深度优先遍历（DFS）
- 4、实现连通图 G（以邻接矩阵存储）的深度优先遍历（DFS）
- 5、设计算法，求无向连通图距顶点 v 最远的一个结点（即路径长度最大）
- 6、求无向图的连通分量个数（DFS/BFS）
- 7、有向图采用邻接表存储，判断顶点 V_i 和顶点 V_j 之间是否存在路径
- 8、在有向图中，如果顶点 r 到图中所有顶点都存在路径，则称 r 为图的根结点。编写代码输出有向图中所有根结点。

查找

- 1、在有序数组中二分查找值为 key 的元素
- 2、寻找二叉排序树中最大值和最小值结点【二叉排序树的应用】
- 3、求出值为 key 的结点在二叉排序树的层次【二叉排序树的应用】
- 4、判断给定二叉树是否是二叉(搜索)排序树【树递归的应用】

排序

- 1、直接插入排序
- 2、折半插入排序
- 3、选择排序
- 4、冒泡排序
- 5、快速排序

顺序表默认结构体:

```
#define maxsize 50
typedef struct{
    int data[maxsize];
    int length;
} Sqlist;
```

1、在顺序表 L 的第 k 个位置插入元素 x

```
bool insert_x(Sqlist *L, int k, int x){
    if (k < 1 || k > L->length+1 || L->length >= maxsize)
        return false;
    for (int i = L->length-1; i >= k-1; i--)
        L->data[i+1] = L->data[i];
    L->data[k-1] = x;
    L->length++;
    return true;
} //时空复杂度分别为 O(N)和 O(1) (如果是数组呢?)
```

2、将顺序表中的元素逆置

```
void Reverse(Sqlist *L){
    for (int i = 0, j = L->length - 1; i < j; i++, j--){
        int temp = L->data[i];
        L->data[i] = L->data[j];
        L->data[j] = temp;
    }
} //时空复杂度分别为 O(N)和 O(1)
```

3、查找顺序表中第一个值为 x 的元素的位置 (和索引有区别)

```
int find_x(Sqlist L, int x){
    for (int i = 0; i < L.length; i++)
        if (L.data[i] == x) return i + 1;
    return -1;
} //时空复杂度分别为 O(N)和 O(1)
```

4、删除顺序表中值为 x 的元素 (仅有一个 x)

```
bool del_x(Sqlist *L, int x){
    for (int i = 0; i < L->length; i++)
        if (L->data[i] == x){
            for (int j = i + 1; j < L->length; j++)
                L->data[j - 1] = L->data[j];
            L->length--;
            return true;
        }
    return false;
} //时空复杂度分别为 O(N)和 O(1) (如果有多个 x 呢? 强化会讲)
```

5、顺序表递增有序, 插入元素 x, 仍递增有序 (非递减)

```
void insert_L(Sqlist *L, int x) {
    int pos;
    for (pos = 0; pos < L->length && L->data[pos] < x; pos++);
    for (int i = L->length - 1; i >= pos; i--)
        L->data[i + 1] = L->data[i];
    L->data[pos] = x;
    L->length++;
} //时空复杂度分别为 O(N)和 O(1) (学完二分, 还可以优化)
```

6、删除整个顺序表中最小的元素（若存在多个相同的最小元素，默认选择第一个）

```
void delete_min(SqList *L){
    int min = 0;
    for (int i = 1; i < L->length; i++)
        if (L->data[i] < L->data[min])
            min = i;
    for (int i = min + 1; i < L->length; i++)
        L->data[i - 1] = L->data[i];
    L->length--;
}
```

拓展，将最小元素和第一个元素进行交换呢？

```
void delete_min(SqList *L){
    int min = 0;
    for (int i = 1; i < L->length; i++)
        if (L->data[i] < L->data[min])
            min = i;
    int temp = L->data[0];
    L->data[0] = L->data[min];
    L->data[min] = temp;
}
```

单链表默认结构体：

```
typedef struct LNode{
    int data;
    struct LNode *next;
} LNode, *LinkList;
```

1、对带头结点的单链表 L 头插/尾插一个值为 x 的节点

```
void headInsert(LinkList L, int x){
    LinkList s = (LinkList)malloc(sizeof(LNode));
    s->data = x;
    s->next = L->next;
    L->next = s;
}
```

```
void tailInsert(LinkList L, int x){
    LinkList s = (LinkList)malloc(sizeof(LNode));
    s->data = x;
    s->next = NULL;
    LinkList p = L;
    for (p = L; p->next; p = p->next);
    p->next = s;
}
```

2、查找单链表中第一个值为 x 的结点

```
LinkList search (LinkList L, int x){
    for (LinkList p = L->next; p!=NULL; p = p->next)
        if (p->data == x)
            return p;
    return NULL;
}
```

3、试编写算法将带头结点的单链表就地逆置

```
void Reverse(LinkList L){
    LinkList p = L->next;
    L->next = NULL;
    while (p != NULL){
        LinkList r = p->next;
        p->next = L->next;
        L->next = p;
        p = r;
    }
}
```

} //时空复杂度分别为 $O(N)$ 和 $O(1)$

4、删除单链表中第一个值为 x 的结点

```
bool del(LinkList L, int x){
    for (LinkList r = L; r->next; r = r->next)
        if (r->next->data == x){
            LinkList p = r->next;
            r->next = p->next;
            free(p);
            return true;
        }
}
```

return false;

} //时空复杂度分别为 $O(N)$ 和 $O(1)$

5、对递增单链表 L ，插入一个值为 x 的结点后。使其保持非递减有序

```
void InsertNode(LinkList L, int x) {
    LinkList p = (LinkList)malloc(sizeof(LNode));
    p->data = x;
    p->next = NULL;
    LinkList r;
    for (r = L; r->next && r->next->data < x; r = r->next)
        ;
    p->next = r->next;
    r->next = p;
}
```

//时空复杂度分别为 $O(N)$ 和 $O(1)$

6、删除单链表 L 中最小值结点（多个选择第一个/默认唯一）

```
void delete_min(LinkList L){
    LinkList pos = L;
    for (LinkList p = L; p->next; p = p->next)
        if (p->next->data < pos->next->data)
            pos = p;
    LinkList u = pos->next;
    pos->next = u->next;
    free(u);
}
```

} //时空复杂度分别为 $O(N)$ 和 $O(1)$

拓展，将最小值结点与后继结点交换位置呢？（假设存在后继结点）

```
LinkList u = pos->next;
LinkList r = u->next;
u->next = r->next;
r->next = pos->next;
pos->next = r;
```

7、删除循环单链表 L 中最小值结点（多个选择第一个/默认唯一）

```
void delete_min(LinkList L){
    LinkList pos = L;
    for (LinkList p = L; p->next != L; p = p->next)
        if (p->next->data < pos->next->data)
            pos = p;
    LinkList u = pos->next;
    pos->next = u->next;
    free(u);
} //时空复杂度分别为 O(N)和 O(1)
```

8、按递增有序地输出单链表中各结点的数值，并释放结点所占空间

```
void del_min(LinkList L){
    while (L->next != NULL){
        LinkList pos = L;
        for (LinkList p = L; p->next; p = p->next)
            if (p->next->data < pos->next->data)
                pos = p;
        printf("%d", pos->next->data);
        LinkList u = pos->next;
        pos->next = u->next;
        free(u);
    }
} //时空复杂度分别为 O(N2)和 O(1)
```

双向链表默认结构体

```
typedef struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
} DNode, *DinkList;
```

9、将一个带头节点双向链表中值最小的结点插入到整个链表的最前面

```
void move(DinkList L){
    DinkList pos = L;
    for (DinkList p = L; p->next; p = p->next)
        if (p->next->data < pos->next->data)
            pos = p;
    if (pos != L){
        DinkList u = pos->next;
        pos->next = u->next;
        if (u->next != NULL) //防止最后一个结点值最小
            u->next->prev = pos;
        u->next = L->next;
        L->next = u;
        u->next->prev = u;
        u->prev = L;
    }
} //时空复杂度分别为 O(N)和 O(1)
```

栈的结构体:

```
#define maxsize 50
typedef struct{
    char data[maxsize];
    int top;
} stack;
```

栈的基本操作（非递归遍历会用到）

```
void init(stack* s){
    s->top = -1;
} //初始化
```

```
bool IsEmpty(stack s){
    return s.top == -1;
} //判断栈空
```

```
bool push(stack* s, int x){
    if (s->top == maxsize - 1)
        return false;
    s->data[++s->top] = x;
    return true;
} //入栈
```

```
bool pop(stack* s, int* x){
    if (s->top == -1)
        return false;
    *x = s->data[s->top--];
    return true;
} //出栈
```

```
bool GetTop(stack s, int* x){
    if (s.top == -1)
        return false;
    *x = s.data[s.top];
    return true;
} //获取栈顶元素
```

1、判断字符串是否回文

```
bool is_true(char str[], int n) {
    for (int i = 0; 2 * i < n - 1; i++)
        if (str[i] != str[n - i - 1])
            return false;
    return true;
} //时空复杂度分别为 O(N)和 O(1)
```

2、判断单链表中全部 n 个字符是否回文

```
bool is_true(LinkList L){
    stack s;
    s.top = -1;
    for (LinkList p = L->next; p; p = p->next)
        s.data[++s.top] = p->data;
    for (LinkList p = L->next; p; p = p->next)
        if (p->data != s.data[s.top--])
            return false;
    return true;
} //时空复杂度分别为 O(N)和 O(N)
（最优解咋做？具体见强化篇）
```

3、判断一个表达式中括号是否配对

```
bool fun(char A[], int n){
    char st[];
    int top = -1;
    for (int i = 0; i < n; i++)
        if (A[i] == '(' || A[i] == '[' || A[i] == '{')
            st[++top] = A[i];
        else if (A[i] == ')' || A[i] == ']' || A[i] == '}')
            if(top == -1)
                return false;
            char c = st[top--];
            if ((A[i]=='&&c!='(') || (A[i]=='&&c!='{') || (A[i]=='&&c!='['))
                return false;
    }
    return top == -1
} //时空复杂度分别为 O(N)和 O(N)
```

队列的结构体

```
typedef struct{
    char data[maxsize];
    int front, rear;
} queue;
```

队列的基本操作（层次遍历会用到）

```
void init(queue* q){
    q->front = q->rear = 0;
} //初始化

bool IsEmpty(queue q){
    return q.front == q.rear;
} //判断队空

bool enqueue(queue* q, int x){
    if ((q->rear + 1) % maxsize == q->front)
        return false;
    q->data[q->rear] = x;
    q->rear = (q->rear + 1) % maxsize;
    return true;
} //入队

bool dequeue(queue* q, int* x){
    if (q->front == q->rear)
        return false;
    *x = q->data[q->front];
    q->front = (q->front + 1) % maxsize;
    return true;
} //出队
```


4、判断子串 **s2** 是否匹配母串 **s1**。若匹配，输出匹配到第一个字符的索引，否则输出-1。

```
int find(char s1[], char s2[]) {
    for (int i = 0; s1[i]; i++) {
        int p1 = i, p2 = 0;
        while (s1[p1] && s2[p2] && s1[p1] == s2[p2]){
            p1++;
            p2++;
        }
        if ( !s2[p2] )
            return i; // 匹配成功
    }
    return -1; // 匹配失败
}
```

5、有两个链表 **A** 和 **B**，判断 **B** 是否是 **A** 的连续子序列

```
bool find(LinkList A, LinkList B){
    for (LinkList q= A->next; q; q = q->next){
        LinkList p1 = q;
        p2 = B->next;
        while (p1 && p2 && p1->data == p2->data){
            p1 = p1->next;
            p2 = p2->next;
        }
        if (!p2)
            return true;
    }
    return false;
}
```

二叉树的结构体 (链式存储) (二叉链表存储)

```
typedef struct BTreeNode{
    char data;
    struct BTreeNode *lchild, *rchild;
} BTreeNode, *BiTree;
```

1、计算二叉树中所有结点个数

```
int count1(BiTree p){
    if (p == NULL)
        return 0;
    else{
        int n1 = count1(p->lchild);
        int n2 = count1(p->rchild);
        return n1 + n2 + 1;
    }
}
```

操作型:

```
void count(BiTree p, int* n){
    if (p != NULL){
        ++(*n);
        count(p->lchild, n);
        count(p->rchild, n);
    }
} //时空复杂度分别为 O(N)和 O(N)
```

2、计算二叉树中所有叶子结点的个数

计算型:

```
int count2(BiTree p){
    if (p == NULL)
        return 0;
    if (!p->lchild && !p->rchild)
        return 1;
    int n1 = count2(p->lchild);
    int n2 = count2(p->rchild);
    return n1 + n2;
}
```

操作型:

```
void func(BiTree p, int* n){
    if (p != NULL){
        if (!p->lchild && !p->rchild)
            ++(*n);
        func(p->lchild, n);
        func(p->rchild, n);
    }
} //时空复杂度分别为 O(N)和 O(N)
```

3、计算二叉树中所有双分支的结点个数

计算型:

```
int func(BiTree p){
    if (p == NULL)
        return 0;
    int n1 = func(p->lchild);
    int n2 = func(p->rchild);
    if (p->lchild && p->rchild)
        return n1 + n2 + 1;
    else
        return n1 + n2;
}
```

操作型:

```
void count(BiTree p, int* n){
    if (p != NULL){
        if (p->lchild && p->rchild)
            ++(*n);
        count(p->lchild, n);
        count(p->rchild, n);
    }
} //时空复杂度分别为 O(N)和 O(N)
```

//如果是单分支结点个数呢?

4、找出二叉树中值最大的结点

操作型:

```
void find(BiTree p, BiTree *m){
    if (p != NULL){
        if ((*m) == NULL || p->data > (*m)->data)
            *m = p;
        func(p->lchild, m);
        func(p->rchild, m);
    }
} //时空复杂度分别为 O(N)和 O(N)
```

5、把二叉树所有结点左右子树交换

操作型:

```
void swap(BiTree p){
    if (p != NULL){
        BiTree temp = p->lchild;
        p->lchild = p->rchild;
        p->rchild = temp;
        swap(p->lchild);
        swap(p->rchild);
    }
} //时空复杂度分别为 O(N)和 O(N)
```

6、求二叉树中值为 x 的层号

计算型：

```
int find(BiTree p, int x) {
    if (p == NULL)
        return 0;
    if (p->data == x)
        return 1;
    int L1 = find(p->lchild, x);
    if (L1 != 0)
        return L1 + 1;
    int L2 = find(p->rchild, x);
    if (L2 != 0)
        return L2 + 1;
    return 0;
}
```

操作型：

```
void find(BiTree p, int x, int L){
    if (p != NULL){
        if (p->data == x)
            printf("%d", L);
        find(p->lchild, x, L + 1);
        find(p->rchild, x, L + 1);
    }
}
```

} //时空复杂度分别为 $O(N)$ 和 $O(N)$

7、计算二叉树的最大深度（高度）

计算型：

```
int find(BiTree p){
    if (p == NULL)
        return 0;
    else{
        int L1 = find(p->lchild);
        int L2 = find(p->rchild);
        return (L1 > L2 ? L1 : L2) + 1;
    }
}
```

操作型：

```
void find(BiTree p, int L, int *max_L){
    if (p != NULL){
        if (L >= *max_L)
            *max_L = L;
        find(p->lchild, L + 1, max_L);
        find(p->rchild, L + 1, max_L);
    }
}
```

} //时空复杂度分别为 $O(N)$ 和 $O(N)$

8、判断两个二叉树是否相似

计算型：

```
bool simi(BiTree p1, BiTree p2){
    if (p1 == NULL && p2 == NULL)
        return true;
    if (p1 == NULL || p2 == NULL)
        return false;
    bool left = simi(p1->lchild, p2->lchild);
    bool right = simi(p1->rchild, p2->rchild);
    return left && right;
}
```

9、求树的度（树中结点度取最大值为树的度）

操作型：

```
void func(BiTree p, int *degree){
    if (p != NULL){
        int temp;
        if (p->lchild && p->rchild)
            temp = 2;
        else if (p->lchild || p->rchild)
            temp = 1;
        else
            temp = 0;
        if (temp > *degree)
            *degree = temp;
        func(p->lchild, degree);
        func(p->rchild, degree);
    }
}
```

10、判断二叉树是不是正则二叉树（即每个结点的度均为 0 或 2）

计算型：

```
bool func(BiTree p) {
    if (p == NULL)
        return true;
    if ((!p->lchild && p->rchild) || (p->lchild && !p->rchild))
        return false;
    bool left = func(p->lchild);
    bool right = func(p->rchild);
    return left && right;
}
```

操作型：

```
void func(BiTree p, bool *flag){
    if (p != NULL){
        if ((!p->lchild && p->rchild) || (p->lchild && !p->rchild))
            *flag = false;
        func(p->lchild, flag);
        func(p->rchild, flag);
    }
} //时空复杂度分别为 O(N)和 O(N)
```

11、层次遍历二叉树

```
void level(BiTree p){
    BiTree que[maxsize];
    int front = 0, rear = 0;
    if (p != NULL){
        que[rear++] = p;
        while (front != rear){
            p = que[front++];
            printf("%d", p->data);
            if (p->lchild != NULL)
                que[rear++] = p->lchild;
            if (p->rchild != NULL)
                que[rear++] = p->rchild;
        }
    }
}
```

} //时空复杂度分别为 O(N)和 O(N)

拓展：试给出自下而上从右到左的层次遍历？

图的结构体

邻接表存储:

```
typedef struct ANode{
    int adjvex;
    struct ANode *nextarc;
} ANode, *Node; //边结点结构体
```

```
typedef struct{
    int data;
    ANode *firstarc;
} Vnode; //顶点结构体
```

```
typedef struct{
    int numver, numedg;
    Vnode adjlist[maxsize];
} Graph;
```

邻接矩阵存储:

```
typedef struct{
    int numver, numedg;
    int verticle[maxsize];
    int Edge[maxsize][maxsize];
} mGraph;
```

1、实现连通图 G (以邻接表存储) 的广度优先遍历 (BFS)

```
void BFS(Graph G, int v){
    int visit[maxsize] = {0}, que[maxsize], front = 0, rear = 0;
    que[rear++] = v;
    visit[v] = 1;
    while (front != rear){
        v = que[front++];
        printf("%d", v);
        for (Node p = G.adjlist[v].firstarc; p; p = p->nextarc)
            if (visit[p->adjvex] == 0){
                que[rear++] = p->adjvex;
                visit[p->adjvex] = 1;
            }
    }
}
```

} //本题时空复杂度分别为 $O(V+E)$ 和 $O(V)$

2、实现连通图 G (以邻接矩阵存储) 的广度优先遍历 (BFS)

```
void BFS(mGraph G, int v){
    int visit[maxsize] = {0}, que[maxsize], front = 0, rear = 0;
    que[rear++] = v;
    visit[v] = 1;
    while (front != rear){
        v = que[front++];
        printf("%d", v);
        for (int i = 0; i < G.numver; i++)
            if (G.Edge[v][i] == 1 && visit[i] == 0){
                que[rear++] = i;
                visit[i] = 1;
            }
    }
}
```

} //时空复杂度分别为 $O(V^2)$ 和 $O(V)$

3、实现连通图 G（以邻接表存储）的深度优先遍历（DFS）

```
void DFS(Graph G, int v, int visit[]){
    printf("%d", v);
    visit[v] = 1;
    for (Node p = G.adjlist[v].firstarc; p != NULL; p = p->nextarc)
        if (visit[p->adjvex] == 0)
            DFS(G, p->adjvex, visit);
} //时空复杂度分别为 O(V+E)和 O(V)
```

4、实现连通图 G（以邻接矩阵存储）的深度优先遍历（DFS）

```
void DFS(mGraph G, int v, int visit[]){
    printf("%d", v);
    visit[v] = 1;
    for (int i = 0; i < G.numver; i++)
        if (G.Edge[v][i] == 1 && visit[i] == 0)
            DFS(G, i, visit);
} //时空复杂度分别为  $O(V^2)$ 和  $O(V)$ 
```

5、求无向连通图距顶点 v 最远的一个结点（即路径长度最大）

```
int BFS(Graph G, int v){
    int visit[maxsize] = {0}, que[maxsize], front = 0, rear = 0;
    que[rear++] = v;
    visit[v] = 1;
    while (front != rear){
        v = que[front++];
        for(Node p = G.adjlist[v].firstarc; p; p = p->nextarc)
            if (visit[p->adjvex] == 0){
                que[rear++] = p->adjvex;
                visit[p->adjvex] = 1;
            }
    }
    return v;
} //时空复杂度分别为 O(V+E)和 O(V)
```

6、求无向图的连通分量个数

```
int func(Graph G){
    int visit[maxsize] = {0};
    int count = 0;
    for (int v = 0; v < G.numver; v++)
        if (visit[v] == 0){
            DFS(G, v, visit); //或者 BFS(G, v)
            count++;
        }
    return count;
} //邻接表的时空复杂度分别为  $O(V*(V+E))$ 和  $O(V)$ 
//邻接矩阵的时空复杂度分别为  $O(V^3)$ 和  $O(V)$ 
```

7、有向图采用邻接表存储，判断顶点 V_i 和顶点 V_j 之间是否存在路径

```
void DFS(Graph G, int i, int j, bool* res, int visit[]){
    if (i == j){
        *res = true;
        return;
    }
    visit[i] = 1;
    for (Node p = G.adjlist[i].firstarc; p != NULL; p = p->nextarc)
        if (visit[p->adjvex] == 0)
            DFS(G, p->adjvex, j, res, visit);
} //法一：DFS
```

```
bool BFS(Graph G, int i, int j){
    int visit[maxsize] = {0}, que[maxsize], front = 0, rear = 0;
    que[rear++] = i;
    visit[i] = 1;
    while (front != rear){
        i = que[front++];
        if (i == j)
            return true;
        for (Node p = G.adjlist[i].firstarc; p; p = p->nextarc){
            if(visit[p->adjvex] == 0){
                que[rear++] = p->adjvex;
                visit[p->adjvex] = 1;
            }
        }
    }
    return false;
} //法二：BFS
```

8、在有向图中，如果顶点 r 到图中所有顶点都存在路径，则称 r 为图的根结点。编写代码输出有向图中所有根结点。

```
void func(Graph G){
    for(int i = 0; i < G.numver; i++){
        int visit[maxsize] = {0};
        DFS(G, i, visit);
        bool flag = false;
        for (int j = 0; j < G.numver; j++)
            if (visit[j] == 0)
                flag = true;
        if (flag)
            printf("%d", i);
    }
} //时空复杂度分别为  $O(V*(V+E))$  和  $O(V)$ 
```

需要所有题目及对应讲解视频，可以扫码购买：



1、在有序数组中二分查找值为 **key** 的元素

```
int binsearch(int A[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (A[mid] == key)
            return mid;
        else if (A[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
//非递归，时空复杂度分别为  $O(\log N)$ 和  $O(1)$ 
```

```
int binsearch(int A[], int key, int low, int high){
    if (low > high)
        return -1;
    int mid = (low + high) / 2;
    if (A[mid] == key)
        return mid;
    else if (A[mid] < key)
        return binsearch(A, key, mid + 1, high);
    else
        return binsearch(A, key, low, mid - 1);
}
//递归，时空复杂度分别为  $O(\log N)$ 和  $O(\log N)$ 
```

由于每次递归调用消耗的栈空间与递归深度成正比，因此空间复杂度也是 $O(\log N)$

2、寻找二叉排序树中最大值和最小值结点

```
BiTree Min(BiTree p){
    while (p->lchild)
        p = p->lchild;
    return p;
}
BiTree Max(BiTree p){
    while (p->rchild)
        p = p->rchild;
    return p;
}
//时空复杂度分别为  $O(\log N)$ 和  $O(1)$ 
```

3、求出值为 **key** 的结点在二叉排序树的层次

```
int level(BiTree p, int key){
    int L = 1;
    while (p != NULL)
        if (p->data == key)
            return L;
        else if (p->data > key){
            p = p->lchild;
            L++;
        }
        else{
            p = p->rchild;
            L++;
        }
    return -1;
}
//时空复杂度分别为  $O(\log N)$ 和  $O(1)$ 
```


4、判断给定二叉树是否是二叉(搜索)排序树

```
void inprint(BiTree p, int res[], int *n) {
```

```
    if (T != NULL) {  
        inprint(T->lchild, res, n);  
        res[(*n)++] = T->data;  
        inprint(T->rchild, res, n);  
    }
```

```
}
```

```
bool Is_ture(int res[], int n){
```

```
    for(int i = 0; i < n - 1; i++)  
        if(res[i] >= res[i+1])  
            return false;  
    return true;
```

```
} //法一， 时空复杂度分别为  $O(N)$ 和  $O(N)$ 
```

```
void Is_BST(BiTree p, int *prev, bool *flag) {
```

```
    if (p != NULL && *flag){  
        Is_BST(p->lchild, prev, flag);  
        if (p->data <= *prev)  
            *flag = false;  
        *prev = p->data;  
        Is_BST(p->rchild, prev, flag);  
    }
```

```
} //法二， 时空复杂度分别为  $O(N)$ 和  $O(1)$ 
```

1、直接插入排序

```
void InsertSort(int A[], int n){
```

```
    int j;  
    for (int i = 2; i <= n; i++){  
        A[0] = A[i];  
        for (j = i - 1; A[0] < A[j]; j--)  
            A[j+1] = A[j];  
        A[j+1] = A[0];  
    }
```

```
} //时空复杂度分别为  $O(N^2)$ 和  $O(1)$ 
```

2、折半插入排序

```
void BinInsert(int A[], int n){
```

```
    int j, low, high, mid;  
    for (int i = 2; i <= n; i++){  
        A[0] = A[i];  
        low = 1; high = i-1;  
        while (low <= high){  
            mid = (low + high) / 2;  
            if (A[mid] > A[0])    high = mid - 1;  
            else    low = mid + 1;  
        }  
        for (j = i-1; j >= low; j--)  
            A[j+1] = A[j];  
        A[low] = A[0];  
    }
```

```
} //时空复杂度分别为  $O(N^2)$ 和  $O(1)$ 
```

3、选择排序

```
void SelectSort(int A[], int n){
    for (int i = 0; i < n; i++){
        int pos = i;
        for (int j = i + 1; j < n; j++){
            if (A[pos] > A[j])
                pos = j;
        }
        int temp = A[i];
        A[i] = A[pos];
        A[pos] = temp;
    }
} //时空复杂度分别为  $O(N^2)$ 和  $O(1)$ 
```

4、冒泡排序

```
void swap(int *x, int *y){
    int temp = *x;  *x = *y;  *y = temp;
}

void BubbleSort(int A[], int n){
    for (int i = n-1; i > 0; i--){
        int flag = 0;
        for (int j = 1; j <= i; j++){
            if (A[j-1] > A[j]){
                swap (&A[j-1], &A[j]);
                flag = 1;
            }
        }
        if (flag == 0)
            return;
    }
} //时空复杂度分别为  $O(N^2)$ 和  $O(1)$ 
```

5、快速排序

```
int Partition(int A[], int low, int high){
    int pivot = A[low];
    while (low < high){
        while (low < high && A[high] >= pivot)
            high--;
        A[low] = A[high];
        while (low < high && A[low] < pivot)
            low++;
        A[high] = A[low];
    }
    A[low] = pivot;
    return low;
}

void QuickSort(int A[], int low, int high){
    if (low < high){
        int pos = Partition(A, low, high);
        QuickSort(A, low, pos - 1);
        QuickSort(A, pos + 1, high);
    }
} //时空复杂度分别为  $O(N\log N)$ 和  $O(\log N)$ 
```

